Haskell mínimo para STM

Fuentes: teórica 06-stm (la parte de Haskell), apunte de la práctica 5, guía warm-up. Es lo **justo** para hacer el warm-up y después la Guía 5. No es un curso de Haskell.

---

0. Cómo correr las cosas

Con GHCup instalado (y una vez `cabal update` + `cabal install --lib stm async random`, para la Guía 5):

```
ghci archivo.hs      -- abre el intérprete con el archivo cargado
runghc archivo.hs    -- ejecuta main sin compilar
ghc archivo.hs       -- compila a archivo.exe
```

Adentro de ghci:

Comando	Qué hace
`:r`	recarga el archivo (lo usás todo el tiempo)
`:t expr`	te dice el tipo de algo. **Tu mejor amigo.** `:t map`
`:q`	salir
`longitud [1,2,3]`	evalúa y muestra el resultado

Flujo de trabajo: editás `guias/guia5/guia5-warmup.hs` en VS Code, en ghci hacés `:r` y probás.

---

1. Funciones y tipos

```
doble :: Int -> Int        -- firma: "doble recibe un Int y devuelve un Int"
doble x = x * 2            -- definición. Sin paréntesis ni return: el cuerpo ES el resultado.

suma :: Int -> Int -> Int  -- dos parámetros: lo último después de la última flecha es lo que devuelve
suma x y = x + y

suma 3 4                   -- se llama SIN paréntesis ni comas
doble (suma 3 4)           -- los paréntesis solo agrupan
```

- **Variables de tipo** (minúscula): `longitud :: [a] -> Int` significa "una lista de **cualquier** cosa".
- **Restricciones** (`=>`): `pertenece :: Eq a => a -> [a] -> Bool` significa "cualquier tipo `a`, siempre que se pueda comparar con `==`". `Ord a` = se puede comparar con `<`. Si usás `==` sobre `a` y no ponés `Eq a`, no compila.
- Todo es inmutable. No hay `x = x + 1`: hay valores nuevos.

2. Listas

```
[]             -- lista vacía
[1,2,3]        -- es azúcar para 1 : 2 : 3 : []
x : xs         -- "x pegado adelante de la lista xs" (x es UN elemento, xs es una LISTA)
xs ++ ys       -- concatenar dos listas (también Strings: String = [Char])
```

Toda lista es **o vacía `[]`, o un primer elemento pegado a otra lista `(x:xs)`**. Esto es lo que usás en el ejercicio 1.

3. Pattern matching + recursión (el Ejercicio 1) ⭐

Una función se define **por casos**, uno por línea, según la *forma* del argumento. Se prueban de arriba hacia abajo.

```
producto :: [Int] -> Int
producto []     = 1                   -- caso base: lista vacía
producto (x:xs) = x * producto xs     -- caso recursivo: primer elemento x, resto xs
```

La receta para pensar cualquier función recursiva sobre listas: 1. ¿Qué devuelvo para `[]`? (casi siempre es obvio: 0, 1, False, [], ...) 2. Para `(x:xs)`: **suponé que la función ya anda para `xs`** (no pienses cómo, confiá). ¿Cómo combino `x` con ese resultado para obtener la respuesta de la lista entera?

Ejemplo pensado: `producto [2,3,4]`. Si confío en que `producto [3,4] = 12`, la respuesta es `2 * 12`. Listo, esa es la línea recursiva.

Guardas (`|`): condiciones dentro de un caso. `otherwise` = "si no, ...".

```
-- hastaElPrimero e xs: los elementos de xs antes de la primera aparición de e
hastaElPrimero :: Eq a => a -> [a] -> [a]
hastaElPrimero _ [] = []                      -- _ = "no me importa este argumento"
hastaElPrimero e (x:xs)
  | x == e    = []                            -- encontré: corto, NO sigo la recursión
  | otherwise = x : hastaElPrimero e xs       -- no es: me quedo con x y sigo con el resto
```

Fijate dos trucos ahí que te sirven para `pertenece` y `quitar`: - "Cortar" = simplemente **no** llamar a la recursión en ese caso. - "Conservar el elemento y seguir" = `x : (recursión sobre xs)`.

También existe `if c then a else b` (el `else` es **obligatorio**) y `case`:

```
signo n = if n < 0 then "neg" else "no neg"
```

4. Funciones de alto orden (Ejercicio 2, opcional)

```
map (\x -> x * 2) [1,2,3]       -- [2,4,6].  \x -> ... es una función anónima (lambda)
filter (\x -> x > 0) [1,-2,3]   -- [1,3]
foldr (\x acc -> x + acc) 0 xs  -- suma: recorre de derecha a izquierda acumulando
```

`foldr f base lista` es exactamente la receta del punto 3 empaquetada: `base` es el caso `[]`, y `f x acc` es "cómo combino `x` con el resultado del resto (`acc`)".

Operadores que vas a ver en código ajeno: - `f $ g x` = `f (g x)` (ahorra paréntesis). - `(f . g) x` = `f (g x)` (composición). - `x `div` y`: comillas invertidas convierten una función en operador. `/` es para `Double`; para `Int` usá `div`.

5. Maybe: "puede no haber resultado" (Ejercicio 4)

```
data Maybe a = Nothing | Just a      -- ya viene definido, es así

safeDiv :: Int -> Int -> Maybe Int   -- (es tu dividir del 4c, ojo)
...
```

Para **usar** un `Maybe` se mira qué forma tiene con `case`:

```
describir :: Maybe Int -> String
describir m = case m of
  Nothing -> "no hay"
  Just n  -> "hay un " ++ show n     -- show: convierte a String
```

6. Tipos propios: `data` y records (Ejercicio 3 y 4d)

```
data Color = Rojo | Verde                -- alternativas (como un enum)
data Arbol a = Hoja | Nodo (Arbol a) a (Arbol a)   -- alternativas con datos adentro

-- se usan con pattern matching, igual que las listas:
tamanio :: Arbol a -> Int
tamanio Hoja           = 0
tamanio (Nodo i _ d)   = tamanio i + 1 + tamanio d
```

Records (campos con nombre, se usan **mucho** en STM para agrupar TVars):

```
data Cuenta = Cuenta { titular :: String, saldo :: Int }

c = Cuenta { titular = "Ana", saldo = 100 }   -- o también: Cuenta "Ana" 100
saldo c                                        -- 100: el nombre del campo es una función
c { saldo = 50 }                               -- una cuenta NUEVA igual a c pero con saldo 50
```

Para que ghci pueda mostrar tus tipos, agregá `deriving Show` al final del `data`.

7. Mónadas, en lo justo y necesario (Ejercicios 5 y 7)

Una mónada `m` es un "contexto" con dos operaciones: - `return :: a -> m a`: mete un valor puro en el contexto. **No es el return de Java: no corta nada**, solo envuelve. - `>>= :: m a -> (a -> m b) -> m b`: "corré esto, y con su resultado corré lo siguiente".

La notación **`do`** es azúcar para encadenar `>>=`. Se lee como código imperativo:

```
-- con Maybe: si cualquier paso da Nothing, todo da Nothing (corta solo)
ejemplo :: Maybe Int
ejemplo = do
  a <- safeDiv 100 5        -- <- "sacá el valor de adentro" (acá, 20)
  b <- safeDiv a 0          -- Nothing → el do entero devuelve Nothing, no se sigue
  return (a + b)
```

Las reglas del `do` (valen igual para Maybe, IO y STM):

Escribís	Significa
`x <- accion`	ejecutá la acción y llamá `x` a su resultado
`let y = expr`	`expr` es un valor común (no una acción): solo le ponés nombre
`accion` sola	ejecutala e ignorá el resultado
última línea	es el resultado de todo el bloque (tiene que ser una acción: típicamente `return algo`)

- **Indentación:** todas las líneas del `do` alineadas en la misma columna.
- `if` dentro de un `do`: las dos ramas tienen que ser acciones. Si una rama no hace nada: `else return ()`. Si una rama hace varias cosas: `then do ...` (un sub-bloque).
- `()` (unit) es el "void": `IO ()` = acción que no devuelve nada útil.

No hay `while`: los loops son recursión. Esto es clave para STM:

```
contarHasta :: Int -> IO ()
contarHasta n = loop 1
  where
    loop i = if i > n
               then return ()
               else do
                 print i
                 loop (i + 1)     -- "volver al principio del while"
```

Combinadores ya hechos (Ejercicio 7 te pide escribirlos, después los usás en todos lados):

Función	Equivale a
`when cond accion`	`if (cond) accion;`
`replicateM_ n accion`	`for i in 1..n: accion`
`mapM_ f xs` / `forM_ xs f`	`for x in xs: f(x)` (descarta resultados)
`mapM f xs`	igual pero junta los resultados en una lista
`forever accion`	`while (true) accion`

8. IO (Ejercicios 8, 9, 10)

```
main :: IO ()
main = do
  putStrLn "¿Nombre?"        -- imprimir String con salto de línea
  nombre <- getLine          -- leer una línea: getLine :: IO String
  n <- readLn                -- leer un número (Haskell deduce el tipo por el uso)
  print (n + 1)              -- print = putStrLn . show (sirve para cualquier cosa mostrable)
  putStrLn ("Hola " ++ nombre)
```

La idea más importante (Ejercicio 9): un valor de tipo `IO a` es una **receta**, no algo que ya pasó. `let x = putStrLn "hola"` no imprime nada: le pone nombre a la receta. Recién se ejecuta cuando aparece como paso del `do` que corre `main`. Y se ejecuta **cada vez** que aparece.

Esto es exactamente lo que se usa en STM: `STM a` también es una receta, y `atomically` es lo que la ejecuta.

Errores típicos: - `"Hola " ++ getLine` no compila: `getLine` es una receta `IO String`, no un `String`. Primero `nombre <- getLine`, después `"Hola " ++ nombre`. - `"n vale " ++ n` con `n :: Int` no compila: `++` es entre

Strings → `show n`. - Números negativos como argumento: `f (-1)`, no `f -1`.

Qué hacer del warm-up "a la mínima"

Ordenado por cuánto lo vas a necesitar en la Guía 5:

Ejercicio	¿Hacerlo?	Por qué
1	Sí	recursión + pattern matching: es como escribís loops
4 (sobre todo 4d)	Sí	`Maybe` y records: la Guía 5 ej 1 usa `TVar (Maybe t)`
5b	Sí	`do` con algo que no es IO: es la misma idea que un `do` de STM
7 a, b, d, e	Sí, rápido	`when`, `replicateM_`, `forM_`, `forever` aparecen en todos los ejercicios con threads
8a, 9	Sí	IO y la idea de "receta"
2, 3, 6, 5a/c, 7c, 8b, 10	Opcional	no aparecen en la Guía 5

El esqueleto con las firmas está en `guias/guia5/guia5-warmup.hs`.